

# Technical note for review — week of 2026-08-31

---

**For a colleague reviewing the work, not for the client.** The weekly report ( `weekly_report_2026-08-30` ) is written for Mario and the Google team: plain language, every bond term defined, and addressed to him — "this request can be withdrawn", "this is your decision". This note is the same week written for someone who reads the code.

**What I would like from you** is in §4. The rest is context for it. If you only have twenty minutes, read §3 and §4 and ignore the others.

---

## 1. What changed structurally

Three engines moved into the restructured package, each a verbatim move with the old path left as a shim:

```
pricing/frn.py          -> pricer/core/pricing/floating.py      + assets/corporate/floating
pricing/hybrid.py       -> pricer/core/pricing/hybrid.py      + assets/corporate/hybrid.py
pricing/coupon_schedule.py -> pricer/core/pricing/coupon_schedule.py + assets/corporate/stepped.py
```

Plus one contract change covering all seven instrument types at the JSON endpoint, and the Excel bridge extended to send the three tree products. **302** pytest checks — 223 before the floating/hybrid round, 287 before this week's hardening pass — **48/50** real-Excel checks from 23, and every production driver CSV byte-identical to a pre-change baseline within its own environment.

The migration itself is the least interesting part — it is a move, and it is proven by hashing whole output files rather than by argument. What follows is what the move exposed.

## 2. The context you need for §4

**OAS here is a calibration output, not an input.** Every route solves the spread that makes the model's clean price equal the custodian mark, then computes risk on the calibrated model. So "this bond is not priced" always means "we could not, or would not, solve it" — never "we have no spread for it".

**The refusal culture.** The codebase refuses rather than guesses wherever a silent assumption would produce a *plausible* number: a date sent as a spreadsheet serial, a currency with no usable curve, contradictory exercise terms, a coupon schedule with no numbers in it. Leniency is for presentation — lower-case currency, numbers as text, unsorted schedules; economics are never inferred. That line is the thing I would most want you to test me on.

## 3. Three findings, one pattern

All three surfaced in one week, all are the same failure class, and I think the pattern matters more than any of them individually: **a security or a right that is neither used nor flagged, and therefore appears in no count and produces no message.**

### 3.1 The GBP par-yield file is stored in percent

24 of 26 `*_Yield_Curve.txt` store par yields as decimals; **GBP and DKK store percentages**. The loader multiplied everything by 100, so the 2009-03-31 gilt curve arrived as a 73%–415% par curve, and the bootstrap correctly refused it:

```
Non-positive discount factor at t=3.000 (freq=1);  
par curve is not arbitrage-free at this node.
```

That message is **true about the curve we built and false about the file**. We recorded it as a property of the market data, wrote it into the missing-data registry, and had a replacement-curve request open against two Bloomberg channels for two months. The raw row is `0.731 / 1.183 / 2.341 / 3.157 / 4.157` — that day's gilt market, in percent.

Two bonds were affected. One was flagged; the other was **absent from the output entirely**, and it is a plain fixed-coupon bond, i.e. inside a class already reported complete.

Fixed with an explicit per-file registry, not a sniffer. The reason is DKK: its median value is 0.543, because Danish rates sat near zero for most of the sample. Any "bigger than 1 means percent" rule gets GBP right and DKK wrong — by luck, not by reasoning. A guard now raises `ParYieldUnitError` *before* the bootstrap when a scaled row exceeds 100%.

### 3.2 A callable bond priced by nothing

`TNTD04920858` — 5.00% of 2012-03-01, callable at par from 2011-12-02, a 90-day gap. Held: 850,000 par, 723,542 market value, marked 85.12, A–/A3.

`dataio/universe.py` routed a call gap  $\leq 7$  days to vanilla and excluded the rest as `callable`. `scripts/callable_risk.py` priced only gaps  $> 366$  days. **Two files each owning half of one decision**, with a hole between them. It had even been written down once, in our own WORKLOG, as "1 short-gap callable (32–180d) still unpriced ... minor loose end" — and then fell out of every count that followed.

The fix is not two aligned constants. Responsibilities are now separated: the routing layer decides *whether* a bond is a callable candidate, the driver consumes **every** candidate and applies no threshold of its own, and the tree/wrapper decides whether the contract is *representable*.

### 3.3 An option value of exactly 0.000000 that was not an answer

Verifying 3.2 produced the one I find most instructive. Sent to the lattice, that bond returned an option value of **exactly zero at every volatility**, which reads as *the right is worthless*.

It is not. The lattice exercises on coupon dates and, by construction, never at the root or at maturity. This bond's call sits 90 days before maturity, **inside the final coupon period**, so `call_array` came back entirely `inf` and it priced as a straight bond.

Captured before the fix, on a 3% flat curve where the option is genuinely valuable — continuation at the call date is  $102.5 \cdot \exp(-0.25 \cdot 0.03) = 101.73 > 100$ , so an issuer would call:

|                                            |                |                      |
|--------------------------------------------|----------------|----------------------|
| straight bond                              | 105.5000550571 |                      |
| SAME bond + a call at 2011-12-02           | 105.5000550571 | difference 0.000e+00 |
| same for a put, and for a sinking schedule |                | 0.000e+00            |

**The zero was the absence of a question, not the answer to one.** All eight bonds carrying call terms today were checked and are unaffected, so this was latent rather than live — but I would have reported "the option is worthless" with a straight face.

## The pattern

In each case a *count balanced*. The corporate reconciliation `732 = 565 + 3 + 164` was arithmetically correct with a bond sitting in none of them, because the missing bond carried a named exclusion reason one layer up. So the response is a set-level invariant rather than a total: `dataio/dispositions.reconcile` proves every candidate has exactly one named outcome over **sets of identifiers**, and it distinguishes a **terminal** exclusion reason (a disposition) from a **routing** reason (an instruction, discharged only when the destination honours it). Both drivers run it at run time.

## 4. What I would most like you to challenge

Ordered by how much I would change my mind if you pushed.

### 4.1 Refusing the short-gap callable, rather than pricing it

`TNTD04920858` is now **named but unpriced**: the lattice refuses it because its call lands on no exercise node. The two ways out are both deferred:

- **give the lattice an exercise-only node** between the last coupon and maturity — a real change to the time grid, short-rate calibration and coupon/ex-coupon interaction. It was put to me in review that QuantLib treats callability dates as mandatory lattice times, alongside coupon and redemption times, which would make this a model design rather than a patch. **I have not verified that against QuantLib's source**, and I am flagging it as second-hand rather than repeating it as fact;
- **adopt a documented rule** that a sub-year par call is economically non-callable and route it to vanilla — but that is a *new modelling threshold*, and it is right for this deep discount and wrong in general (a bond above par with an 11-month call has real option value).

I deferred both because the round's scope forbade engine changes and new thresholds. **The counter-argument I find strongest**: we now knowingly hold a security we cannot price, and the first option is maybe half a day. Is "correct and unpriced" actually better than "approximately priced and labelled"? I think yes, but not overwhelmingly.

### 4.2 `ExerciseTermsError` subclasses `ValueError`

Every exercise-terms refusal is now one family carrying the JSON path a caller can act on, because they were being swallowed by the spread solver's `except ValueError` and reported as "no spread reprices this bond — check the price, the coupon and the maturity" — three fields, all of them correct.

I kept it a `ValueError` subclass so existing callers behave. **But that is also exactly why they were swallowed.** A base that is *not* a `ValueError` would make swallowing impossible rather than merely fixed-

in-two-places. I chose compatibility; I am not certain that was right, and the failure has now recurred three times (sinking basis, the grid guard, the call/put conflict).

### 4.3 The disposition invariant lives in the drivers, not in pytest

I put it there deliberately: a pytest version would have to read git-ignored driver output and could pass against a stale file on a machine that had not re-run. Running it in the driver means it cannot go stale.

**The cost:** CI-by-pytest alone does not enforce it. Someone who only runs the test suite gets no coverage of the property that has now failed twice. Is there a shape that gets both?

### 4.4 Synthetic evidence for puttable and sinking

No URS holding is a puttable or a sinking-fund bond, so both routes are validated on invented fixtures plus invariants (put/call mirror symmetry,  $f=1$  reducing to the call cap bit-for-bit, value monotone in the fraction). It is labelled synthetic everywhere it appears.

**Is that enough to say the route works?** I think invariants plus an exact anchor are stronger than a single golden number would be, but it is a claim about the model rather than about the book, and I would rather you disagreed now than after it matters.

### 4.5 The floating-rate duration sign

Two exact regimes, and the sign flips:

| current coupon             | effective duration                                   | why                                                      |
|----------------------------|------------------------------------------------------|----------------------------------------------------------|
| supplied                   | <b>+0.104396</b> = +time to the next reset           | the running coupon is fixed; a curve bump cannot move it |
| projected                  | <b>-0.395604</b> = -time <i>since</i> the last reset | the bump reprices that coupon too                        |
| <i>same-maturity fixed</i> | +17.4381                                             |                                                          |

Both are within one coupon period. A third regime applies to a deep discount, where price  $\approx$  par minus a spread annuity, so a rate rise shrinks the gap and the price rises — that one grows with maturity and with the discount. On the current book the widest is **-1.85** (TNTD04882955, maturing 2016, marked 67.04), against **+6.09** for a same-maturity fixed bond.

(*frn.py* 's docstring still cites  $\approx -10.6$  for a 57-year floater marked near 50. That was true of the June-2026 routing; those 2066/2067 names are fixed-to-float and have since moved to the hybrid engine, so no floater that long remains on this route. I quoted the docstring here before checking the output — the third time this week that rule has caught me, which is why it is the last paragraph of this note.)

I am confident in the arithmetic. What I am less sure of is whether **projecting** the current period's coupon is the right default at all, given that in reality the running coupon *is* fixed and known. We do it because most of this book's cells give no reset history. It makes the reported duration point the wrong way for the majority of holdings.

## 4.6 A stale default on the same axis as 3.3

`dataio/call_schedules.to_lattice_schedule` still defaults to **365.25 days per year**, while the coupon grid it feeds is **ACT/364**:

```
DAYS_PER_YEAR = 365.25
def to_lattice_schedule(date_entries, val_date, days_per_year=DAYS_PER_YEAR):
```

Production is safe — both drivers pass `days_per_year=364.0` explicitly — so this is latent, not live. But it is the **same failure as 3.3 in a different place**: an exercise date landing on the wrong point of the time axis, silently, with a plausible number out the other end. A new caller taking the default gets it.

I left it alone because the round forbade unrelated changes, and because "it is only a default" was how the short-gap callable survived too. **My inclination is to delete the default outright** and make the day count a required argument, so the question cannot be answered by omission. That is a one-line change with a handful of call-site edits. Is there a reason to keep a default at all?

## 4.7 Pricing on provisional call terms

The three callable bonds that *do* price use schedules seeded from the custodian's own first-call-date column, at **par@100** — a v1 convention, approved as such, and explicitly not Bloomberg-confirmed. `data/call_schedules.csv` now carries that provenance per row.

The precedent that worries me is Sempra: its `AB` column turned out to be a first-coupon date, and the bond was make-whole-only with no par call at all. We priced it on the lattice until the ISIN lookup caught it, and the "par-call conflicts with the mark" symptom was visible the whole time and was read as a data oddity rather than as a wrong assumption.

So the question is not whether the convention is reasonable — it is whether **pricing on unverified exercise terms and labelling the output provisional** is better than **flagging those bonds until real terms arrive**. We currently do the former for three securities. I think that is right, because the alternative flags a bond whose option is demonstrably near-worthless and loses a number for no gain — but the Sempra case is a real counter-example, and one of the three is `TNTD04115619`, whose implied spread is 1993.6 bp, which is the kind of number that should make anyone check the terms behind it.

## 5. Where I think the remaining risk is

- **Row/security counting.** The Corporate Bonds tab has 676 rows but 616 unique securities — 60 asset IDs are listed more than once. I had `F13` described as "2 rows, 1 held" in four documents, which reads as "one is not a holding"; it is the same bond twice. Anywhere a count is quoted, the population matters more than the number.
- **The private-helper coupling.** `hybrid` imports `_as_date`, `_rate`, `_df`, `simple_forward` and `YEAR_DAYS` from `floating` by name, so the shim re-exports privates. It is pinned by a test named after the reason, and I deliberately did *not* refactor it this round — but it is the most fragile seam in the migrated code.
- **Two thresholds, one decision.** 3.2 was that shape. I have not swept the codebase for others, and I suspect there are more.

- **Defaulted bonds are treated two ways.** Four securities carry `primary_reason = defaulted`. Two get a `recovery` row in the output at the custodian mark; two do not appear at all. `TNTD03044683` — par 1,450,000, marked 11.25 — is one of the absent ones, while `TNTD03037967` — par 1,625,000, marked 12.00 — gets a row. Both are named in the exclusion log, so neither is invisible in the 3.2 sense, but the treatments should not differ for two bonds in the same situation. I found this during the reconciliation and recorded it rather than changing it, because "make the table tidy" is not a reason to move a route.
- **Five layers are still unmigrated:** `pricing/ilb.py`, `pricing/mbs.py`, `curves/`, `credit/` and `dataio/` are reached through their original paths. The shims are all still in place and nothing has been retired. Worth a view on the order, and on whether the shim layer is permanent or has an end date — it is currently open-ended, which is how a "temporary" compatibility layer becomes the architecture.

## 6. Checking any of this yourself

```
python -m pytest -q                                302 checks, ~24 s
python scripts/column_f_audit.py                    the six-cell audit, all populations
python scripts/calibrate_risk.py                    prints the disposition reconciliation
powershell -File integrations/excel_vba/tests/Run-BridgeTests.ps1      48 checks
powershell -File integrations/excel_vba/tests/Run-BridgeTests.ps1 -PythonExe ... 50, live engine
```

⚠ **One reproduction trap.** Full driver CSVs are byte-identical *within* an environment but not across one: Windows and the Linux server differ by up to **3.6e-8 relative, entirely in the convexity column** — a second difference over the square of a 1 bp bump amplifies a last-bit rounding by  $10^8$ . Text columns match exactly and prices, spreads and durations agree to  $\sim 1e-12$ . If you diff a CSV produced on 47 against one produced on Windows you will see a difference that is **not** a regression. Compare within one machine.

The most load-bearing files for a review:

|                                                             |                                                |
|-------------------------------------------------------------|------------------------------------------------|
| <code>src/dataio/dispositions.py</code>                     | the set-level invariant                        |
| <code>src/pricer/assets/corporate/embedded_option.py</code> | the refusal family and the grid guard          |
| <code>src/curves/bootstrap.py</code>                        | PAR_YIELD_UNITS and the units guard            |
| <code>tests/test_callable_disposition.py</code>             | both defects, pinned                           |
| <code>docs/column_f_delivery_matrix_2026-08-31.md</code>    | the per-security evidence, generated not typed |

**One process note, since it is the rule that keeps catching me.** Building the Excel fixtures, I wrote the expected spreads into the test harness from an earlier single-call test instead of from the fixtures I had just generated with a two-date schedule. Three checks failed at  $\sim 1e-5$  until I read the values out of the files. *Quote from the run, not from memory* — I wrote that rule down last month and broke it **three times this week**: the harness expectations above, the `F13` row/security count that stood in four documents, and the -10.6 duration figure in §4.5 that I lifted from a docstring describing a routing we no longer use. Each was

caught by re-deriving before publishing rather than by anyone reading it. That is an argument for the checking, not for the writing.